



Facultad de Ingeniería
Programación 2

Obligatorio

Integrantes:

Virginia Cota

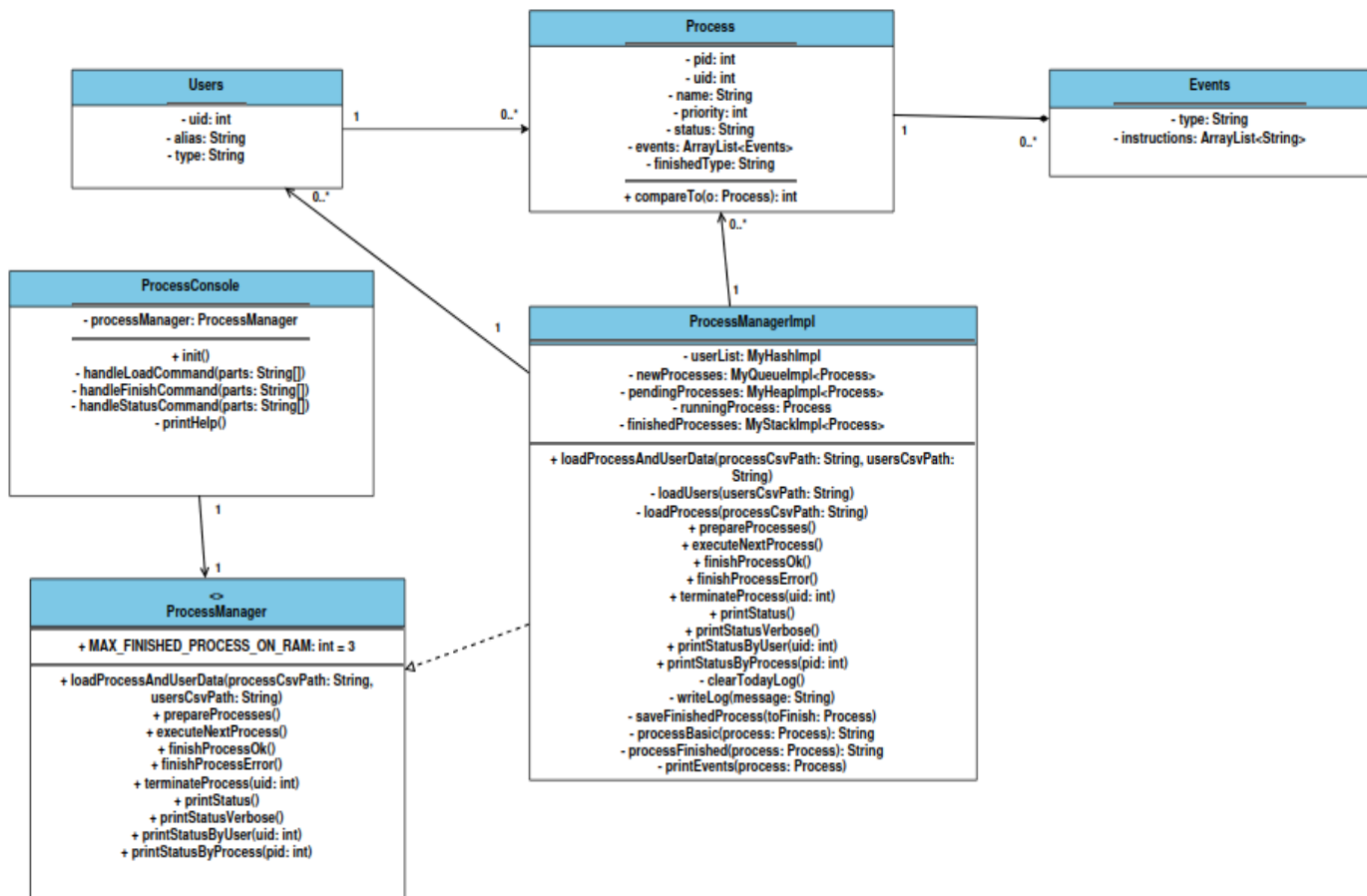
Sebastián Gigirey

Blas Melissari

Manuel Sosa

Semestre 1 de 2026

Diagrama UML



Análisis y justificación de la elección de los TADs y el diseño del modelo.

userList: Decidimos implementarlo mediante un Hash en base al UID pues el UID es un dato único para cada persona, por lo que permite distinguir a un usuario de otro sin ambigüedades. Además es una forma muy eficiente para insertar, buscar y recuperar información.

newProcesses: Decidimos implementarlo como una Queue pues la letra dice “el proceso deberá almacenarse en una **cola** de nuevos procesos “

pendingProcesses: Decidimos implementarlo como un Heap Máximo en base a la prioridad pues como la letra pide “Los procesos pendientes deberán mantenerse ordenados por prioridad “ de esta forma al agregar y quitar procesos siempre queda ordenado de mayor a menor. (Siempre al sacar el siguiente nos va a dar el más alto, es decir el de mayor prioridad)

runningProcess: Es una variable única porque sólo puede existir un proceso en ejecución a la vez. “únicamente podrá existir un proceso en ejecución simultáneamente “

finishedProcesses: Es un Stack pues en la letra dice “deberán almacenarse en memoria dentro de una **pila** de procesos finalizados”

Implementación del sistema de logs

El sistema de logs lo implementamos dentro de la clase **ProcessManagerImpl**, donde utilizamos las clases FileWriter, LocalDate, LocalDateTime, y DateFormatter de Java.

Inicialización del archivo de log

Al crear una instancia de **ProcessManagerImpl**, se ejecuta el modelo:

```
public ProcessManagerImpl() { 2 usages  👤 blasm  
    clearTodayLog();  
}
```

Este método es responsable de crear o reiniciar el archivo de log correspondiente al día actual.

```
private void clearTodayLog() { 1 usage  @ blasn
    String fileName = "DOORS_PROCESS_LOG_" + LocalDate.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd"));

    try (FileWriter writer = new FileWriter(fileName, append: false)) {
        // Vacía el archivo al iniciar el sistema
    } catch (IOException e) {
        System.out.println("Error limpiando log: " + e.getMessage());
    }
}
```

Primero se construye el nombre del archivo utilizando la fecha actual usando *LocalDate*. Luego se crea un *FileWriter* utilizando el parámetro *false*, lo que indica que el archivo debe abrirse en modo de sobrescritura. Esto permite eliminar cualquier contenido previo y comenzar cada ejecución del sistema con un log vacío.

Escritura de registros

La funcionalidad principal del sistema de logs se centralizó en el método *writeLog(String message)*.

```
private void writeLog(String message) { 6 usages  @ blasn
    String fileName = "DOORS_PROCESS_LOG_" + LocalDate.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd"));
    String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));

    try (FileWriter writer = new FileWriter(fileName, append: true)) {
        writer.write( str: "[" + timestamp + "]: " + message + System.lineSeparator());
        writer.flush();
    } catch (IOException e) {
        System.out.println("Error escribiendo log: " + e.getMessage());
    }
}
```

Este método realiza tres tareas principales:

- 1) Obtiene el nombre del archivo correspondiente al día actual.
- 2) Genera una marca temporal (timestamp) con fecha y hora exacta.
- 3) Escribe el mensaje recibido al final del archivo.

En este caso, la creación del *FileWriter* utiliza el parámetro *true*, que habilita el modo *append*. Esto permite agregar nuevas entradas al final del archivo sin eliminar la información ya registrada.

Finalmente se escribe el mensaje con el siguiente formato:

```
writer.write( "[" + timestamp + "]: " + message + System.lineSeparator() );
```

Cada línea almacenada en el log tiene una estructura similar a:

[2026-06-18 10:15:22]: ...: PID=1 ...

De esta forma cada evento queda asociado al momento exacto en que ocurrió.

Integración con el administrador de procesos

Una vez implementado el mecanismo de escritura, el método `writeLog(String message)` fue reutilizado en las distintas operaciones del administrador de procesos.

Por ejemplo, durante la preparación de un proceso (*ppprepare*) se registra la información del proceso que pasa a estado pendiente:

```
writeLog(  
    "NEW PENDING PROCESS: PID=" + toPrepare.getPid()  
    + " | " + toPrepare.getName()  
    + " | USER:" + user.getAlias()  
    + " UID:" + user.getUId()  
    + " | P=" + toPrepare.getPriority()  
);
```

De manera similar, se usa **writeLog(...)**; para el resto de las funciones usando la información necesaria en cada uno.

Manejo de overflow de procesos finalizados

Cuando la stack de procesos finalizados alcanza el límite definido por `MAX_FINISHED_PROCESS_ON_RAM`, se produce una situación de overflow. En este momento, los procesos almacenados en la stack son sacados de memoria y registrados en el archivo de log. Una vez liberado el espacio, el nuevo proceso finalizado es agregado al stack. Esto permite limitar el consumo de memoria manteniendo un historial de los procesos terminados.

Análisis de órdenes de tiempo de ejecución para cada función.

Orden big O de la función **clearTodayLog**: es de orden constante porque solo crea/ abre el archivo log del día actual y lo limpia, $O(1)$.

La función **loadProcessAndUserData** llama a dos subprocesos privados: *loadUsers* y *loadProcess*.

En primer lugar, *loadUsers* utiliza un *BufferedReader* para leer el archivo de usuarios línea por línea. Si llamamos u a la cantidad de usuarios, el *while* se ejecuta u veces. Dentro de ese ciclo, cada usuario se agrega a un Hash, operación que en promedio tiene costo $O(1)$ de inserción. Por lo tanto, el orden de *loadUsers* es $O(u)$.

Por otro lado, *loadProcess* también utiliza un *BufferedReader* para leer el archivo de procesos línea por línea. Si llamamos p a la cantidad de procesos, el *while* se ejecuta p veces. Dentro de cada iteración, hay un *for* que recorre los eventos asociados a ese proceso. Si llamamos e a la cantidad de eventos por proceso, esta parte tiene costo $O(e)$. Además, agregar cada evento al final de un *ArrayList* tiene costo $O(1)$ en promedio, y agregar el proceso a una *Queue* también tiene costo $O(1)$. También verifica que el proceso no esté en la cola con *contains* por lo que eso agrega un p^2 . Por eso, la complejidad de *loadProcess* es $O(p * e + p^2)$.

Por ende en total *loadProcessAndUserData* tiene $O(p * e + p^2 + u)$, o quedandonos con lo mayor $O(n^2)$.

Orden big O de la función **prepareProcesses**: el orden de esta función es de $O(p^2)$ debido a que en la misma se ejecutan un bucle de procesos p , adentro un bucle de eventos e , y además inserciones al heap. , de estos dos órdenes el que presenta mayor crecimiento es el del bucle pues el crecimiento cuadrático es más rápido que el logarítmico lineal.

Orden big O de la función **executeNextProcess**: el orden de esta función es de $O(n)$ pues accede a todo el Array de eventos dentro del proceso a ejecutar $O(n)$ para actualizar el log y saca de un heap con orden $\log(n)$, ($O(n) > O(\log(n))$).

Orden big O de la función **finishProcessOk**: el orden de esta función es de $O(1)$ porque dentro del método accede a la función auxiliar *saveFinishedProcess* que dentro de la misma contiene un bucle *while* de orden n , pero en nuestro caso, técnicamente n es definido 3 por el máximo de procesos definidos. Por ende es $O(1)$.

Orden big O de la función **finishProcessError**: al no incluir un bucle *for* o *while* directo, se podría decir que el orden de esta función es $O(1)$, sin embargo, dentro de esta se llama a la función *saveFinishedProcess* que, como ya mencionamos tiene orden $O(1)$, por tanto el orden "total" de la función resulta $1 * 1$, es decir $O(1)$.

Orden big O de la función **terminateProcess**: el orden de esta función es de $O(1)$ porque dentro del método accede (como en los otros casos) a la función auxiliar *saveFinishedProcess*, y no hay nada más que agregue orden.

Orden big O de la función **printStatus**: es de orden $O(n * \log(n))$ porque dentro de la función se encuentran dos bucles *while* aislados entre sí, pero dentro de estos se sacan todos los

procesos de un heap y se ponen en otro y luego se revierte. Dentro de esta función también se encuentra las funciones auxiliares *processBasic* y *processFinished* que son de orden constante pues solo realizan gets y no tiene bucles.

Orden big O de la función **printStatsVerbose**: tiene orden $O(n \cdot \log(n))$, utiliza un heap auxiliar para recorrer todos los procesos pendientes, sus inserciones o eliminaciones cuestan $\log(n)$, y luego recorre todos los procesos finalizados de forma lineal, entonces resulta $O(n)$. Y al igual que las otras funciones de *printStats* usan funciones auxiliares de orden constante.

Orden big O de la función **printStatsByUser**: tiene orden $O(n \cdot \log(n))$, porque dentro de un bucle while la función inserta datos a un heap auxiliar, y no posee otros órdenes mayores dentro de la función. Y al igual que las otras funciones de *printStats* usan funciones auxiliares de orden constante.

Orden big O de la función **printStatsByProcess**: Al igual que las anteriores, usa un heap auxiliar, entonces tiene $\log(n)$, pero al igual que *printStatsVerbose* recorre los procesos siguientes de forma lineal, entonces se le agrega n , por tanto resulta $O(n \cdot \log(n))$.